CSE370:DATABASE SYSTEMS

BRAC UNIVERSITY
Inspiring Excellence

Lecture 12: Query Optimization

INTRODUCTION

Although the term is optimization, the query is not necessarily optimized. Rather it is improved to the best of the ability of the database system.

Motivation

The SQL query that application programmers/users write may be very inefficient if executed exactly as it is by the database management system. The query may takes a long time to process in its original form and/or consume too much system resources. This is a serious cause of concern as the database management system is expected to answer a lot of queries simultaneously and fast.

Furthermore, even in the original form, different parts of the query can be processed in many different ways having varying costs. An SQL query is a declarative query that states what to retrieve – not how. Hence, the database management system has to decides the best strategy for query execution among many alternatives.

Objectives

The objective of query optimization is to convert the original query into an equivalent query that should perform much better and choose the best execution strategy for the modified query.

GENERAL PRINCIPLES

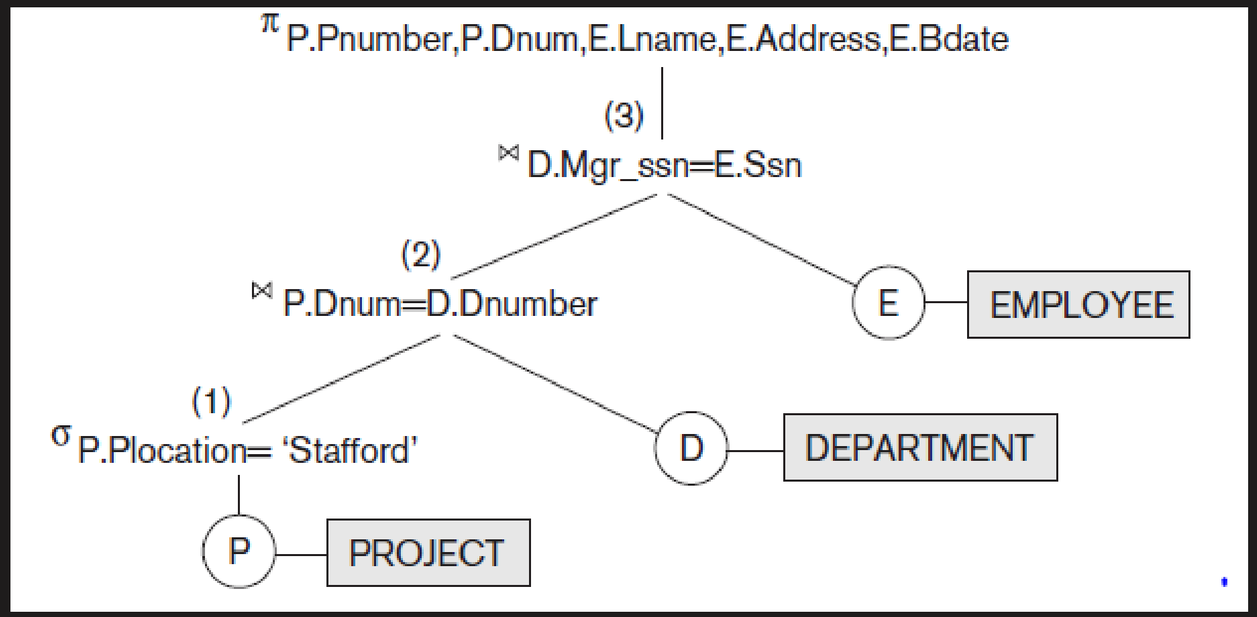
- Reduce the amount of data retrieved from the disk as much as possible.
- Apply filtering of retrieved data as early as possible to avoid consuming too much memory and for too long.
- Order operations in a manner to reduce the amount of computation over data.
- Choose the most cost-effective strategy for executing individual query blocks.
- Reduce disk storage overhead for intermediate results.
- Whenever possible, pipeline processing of query blocks.
- When network is involved, reduce the the number of communication steps among database components and the bandwidth consumption due to message exchanges.

COMPONENTS OF QUERY OPTIMIZATION

- Heuristic Rule-Based Algebraic Optimizations
- Cost-based Optimizations
- Query Implementation Strategy Selection
- Physical Optimization

Query optimization itself is a costly maneuver for the database system. So extensive optimizations are done for frequently used, compiled, queries. For sporadic, interpreted, queries all optimization steps are often not attempted. Furthermore, database systems often cache optimized query execution plans for most important queries.

```
select P.Pnum, P.Dnum, E.Lname, E.Address, E.Bdata
from Emplyee E
        join Department D on D.Mgr_ssn = E.Ssn
        join (select *
                from Project
                where P.Plocation = 'Stafford') P
            on  P.Dnum = D.Dnumber
```



A Query Tree In Relational Algebra

→The database system first analyzes the input SQL query and translate it into its formal relational algebra form.

→Any relational Algebra expression can be represented as a tree.

→ This tree is called the query tree.

→ Improvement opportunities are attempted over this tree as the tree clearly illustrates how to component parts of a query are connected.

Heuristic rule-based algebraic modifications transform the original query tree to another that gives the same result but expected to have lower cost and better performance as the first step of query optimization.

Some Heuristic Rules for Algebraic Modifications

Cascade of σ. A conjunctive selection condition can be broken up into a cascade (that is, a sequence) of individual σ operations:

$$\sigma_{c_1 \text{ AND } c_2 \text{ AND } \ldots \text{ AND } c_n}(R) \equiv \sigma_{c_1}(\sigma_{c_2}(\ldots(\sigma_{c_n}(R))\ldots))$$

Commutativity of σ. The σ operation is commutative:

$$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$$

Cascade of π. In a cascade (sequence) of π operations, all but the last one can be ignored:

$$\pi_{\text{List}_1}(\pi_{\text{List}_2}(\ldots(\pi_{\text{List}_n}(R))\ldots)) \equiv \pi_{\text{List}_1}(R)$$

Commuting σ with π. If the selection condition c involves only those attributes $A_1, \ldots, A_n$ in the projection list, the two operations can be commuted:

$$\pi_{A_1, A_2, \ldots, A_n}(\sigma_c(R)) \equiv \sigma_c(\pi_{A_1, A_2, \ldots, A_n}(R))$$

Commutativity of ⋈ (and ×). The join operation is commutative, as is the × operation:

$$R \bowtie_c S \equiv S \bowtie_c R$$
$$R \times S \equiv S \times R$$

Notice that although the order of attributes may not be the same in the relations resulting from the two joins (or two Cartesian products), the *meaning* is the same because the order of attributes is not important in the alternative definition of relation.

Commuting σ with ⋈ (or ×). If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R—the two operations can be commuted as follows:

$$\sigma_c(R \bowtie S) \equiv (\sigma_c(R)) \bowtie S$$

Commuting σ with set operations. The σ operation commutes with ∪, ∩, and −. If θ stands for any one of these three operations (throughout the expression), we have:

$$\sigma_c(R \,\theta\, S) \equiv (\sigma_c(R)) \,\theta\, (\sigma_c(S))$$

The π operation commutes with ∪.

$$\pi_L(R \cup S) \equiv (\pi_L(R)) \cup (\pi_L(S))$$

Converting a (σ, ×) sequence into ⋈. If the condition c of a σ that follows a × corresponds to a join condition, convert the (σ, ×) sequence into a ⋈ as follows:

$$(\sigma_c(R \times S)) \equiv (R \bowtie_c S)$$

Pushing σ in conjunction with set difference.

$$\sigma_c(R - S) = \sigma_c(R) - \sigma_c(S)$$

However, σ may be applied to only one relation:

$$\sigma_c(R - S) = \sigma_c(R) - S$$

Pushing σ to only one argument in ∩.

If in the condition σ_c all attributes are from relation R, then:

$$\sigma_c(R \cap S) = \sigma_c(R) \cap S$$

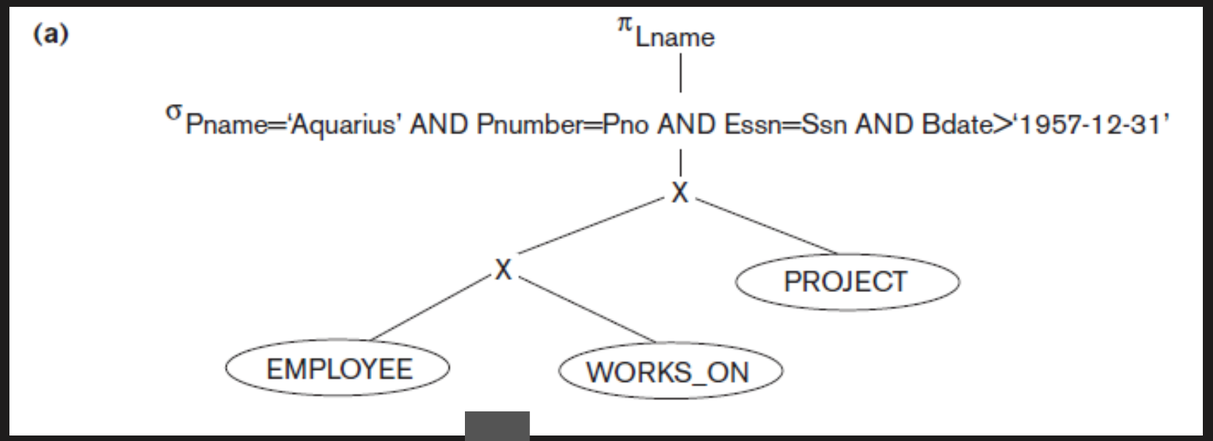
Check the reference textbook for the whole list of modifications

OUTLINE OF ALGEBRAIC MODIFICATION APPLICATION

- Decompose composite select and project operations using cascading rules.
- Move select and project operations downward as much as possible close to the leaves using commutative rules.
- Order the selects in a manner to apply the most restrictive select the first.
- Try to convert any Cartesian product into join operation using selection clauses.
- Identify sub-trees that represents a group of operations that can be performed together using a single algorithm.

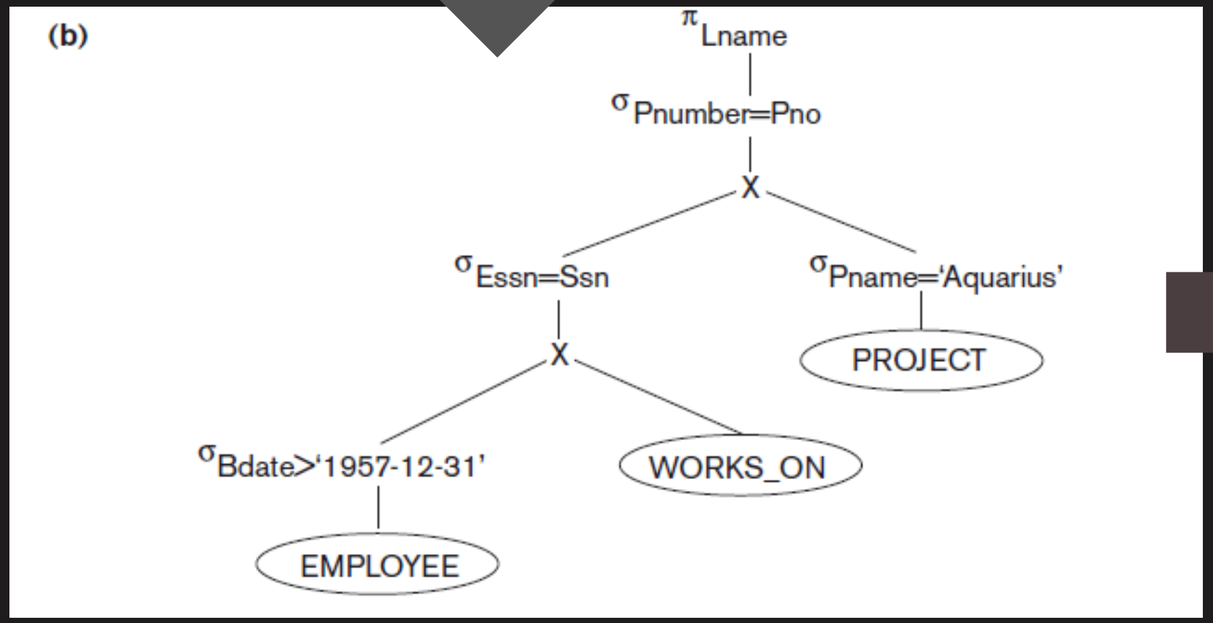
Some modifications may obstruct other modifications. So the query optimization engines has to choose which to apply and which not to based on cost-effectiveness analysis.

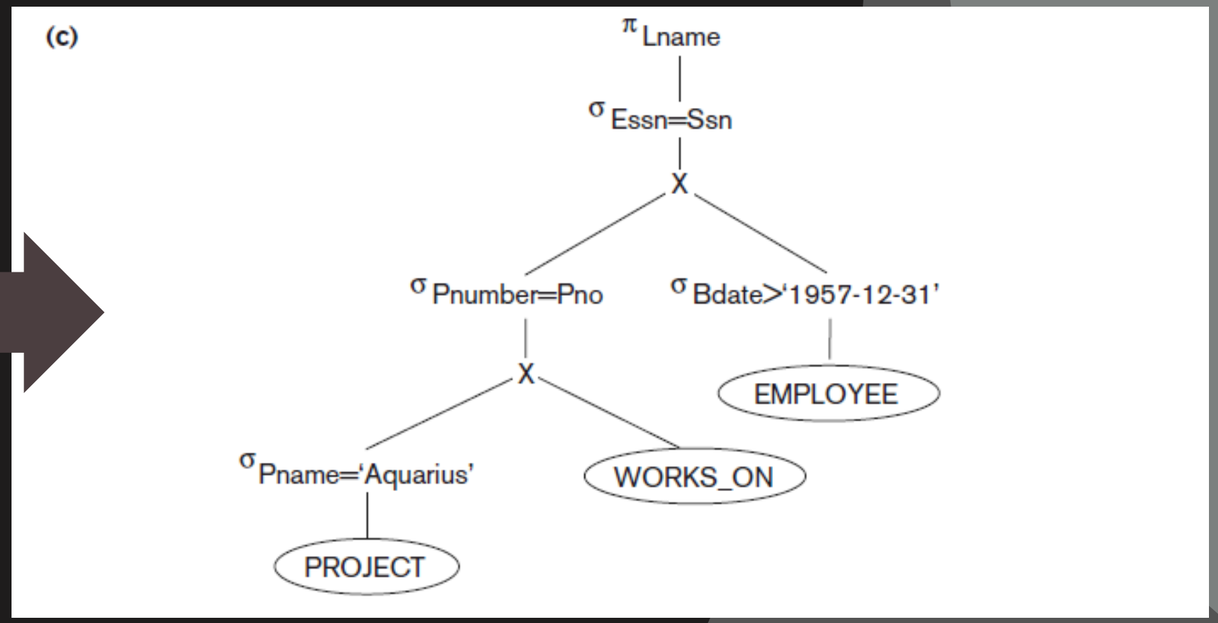
Example: Algebraic Optimization of a Query



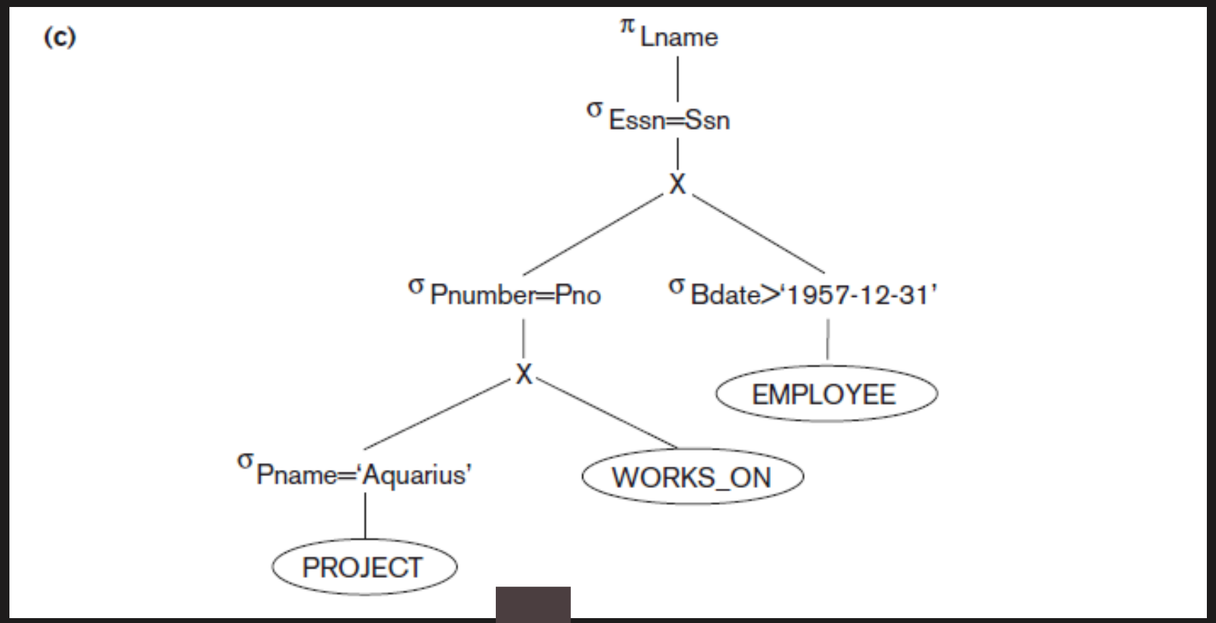
Original Query:

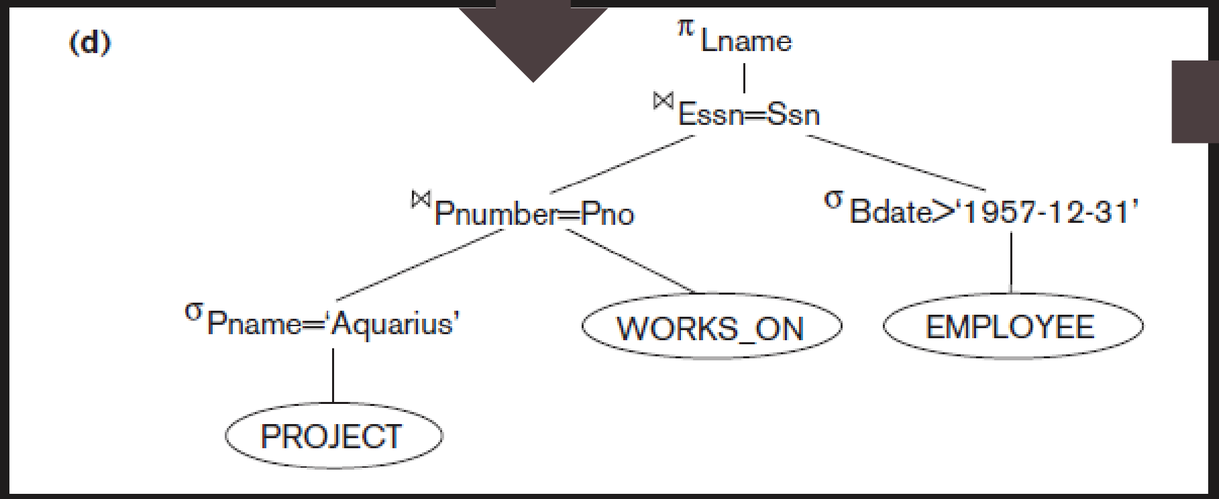
Select Lname
from Employee, Project, Works_On
Where Pname = 'Aquarius'
and Pnumber =Pno
and Essn = Ssn and Bdata > '1957-12-31'





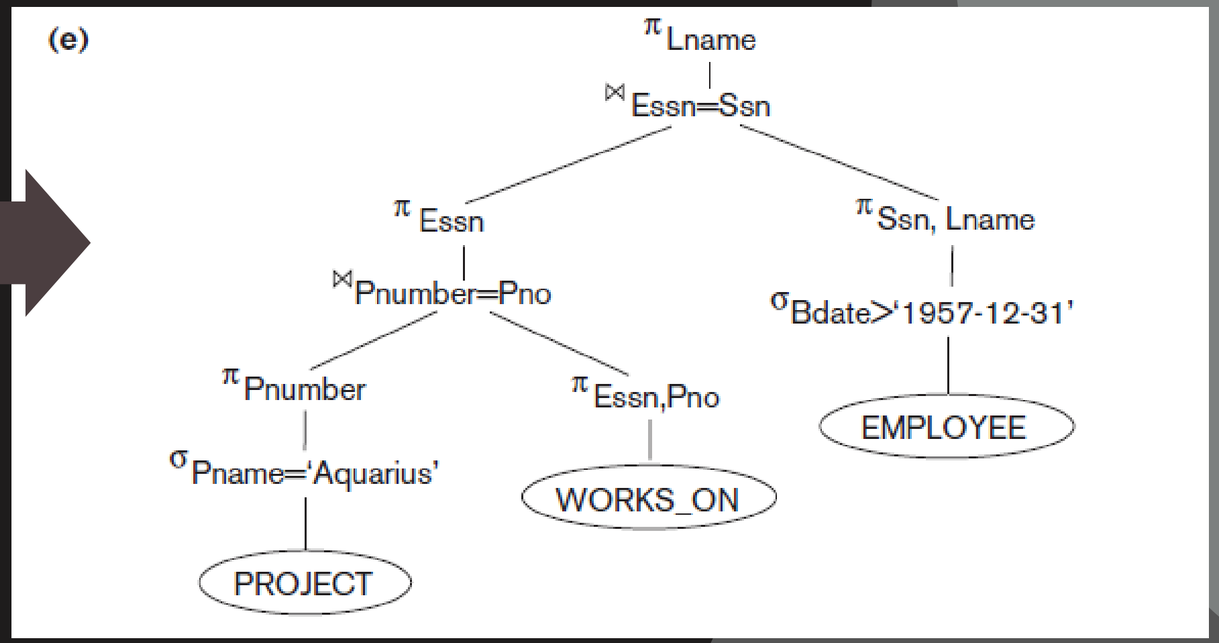
Example: Algebraic Optimization of a Query





Final Query:

```
Select Lname
from  (select Ssn, Lname from Employee where Bdata > '1957-12-31') e
   join (select * from
           (select Pnumber from Project where Pname = 'Aquarius') p
           join (select Essn, Pno from Works_On) w
              on p.Pnumber = w.Pno) pw
      on  pw.Essn = e.ssn
```



Cost-based Optimization Logic

- Heuristic rule-based algebraic modifications of a query typically significantly improve query execution performance. Still, there are many alternative modification candidates with drastically different performance characteristics that heuristics cannot capture.
- The limitation is that algebraic modifications do not take into consideration the actual storage structures of the involved records, existence/non-existence of indexes, and the distribution of records relative to the queried attributes.
- This is where the cost-based query optimization helps. It takes into consideration the storage, memory, computation, and networking costs of executing alternative modified queries and selects the best candidate according to its estimate.
- Furthermore, cost assessments also help to decide the best execution strategies for individual query blocks.

There are two things that database management systems uses for collecting information about data to estimate query execution costs.

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

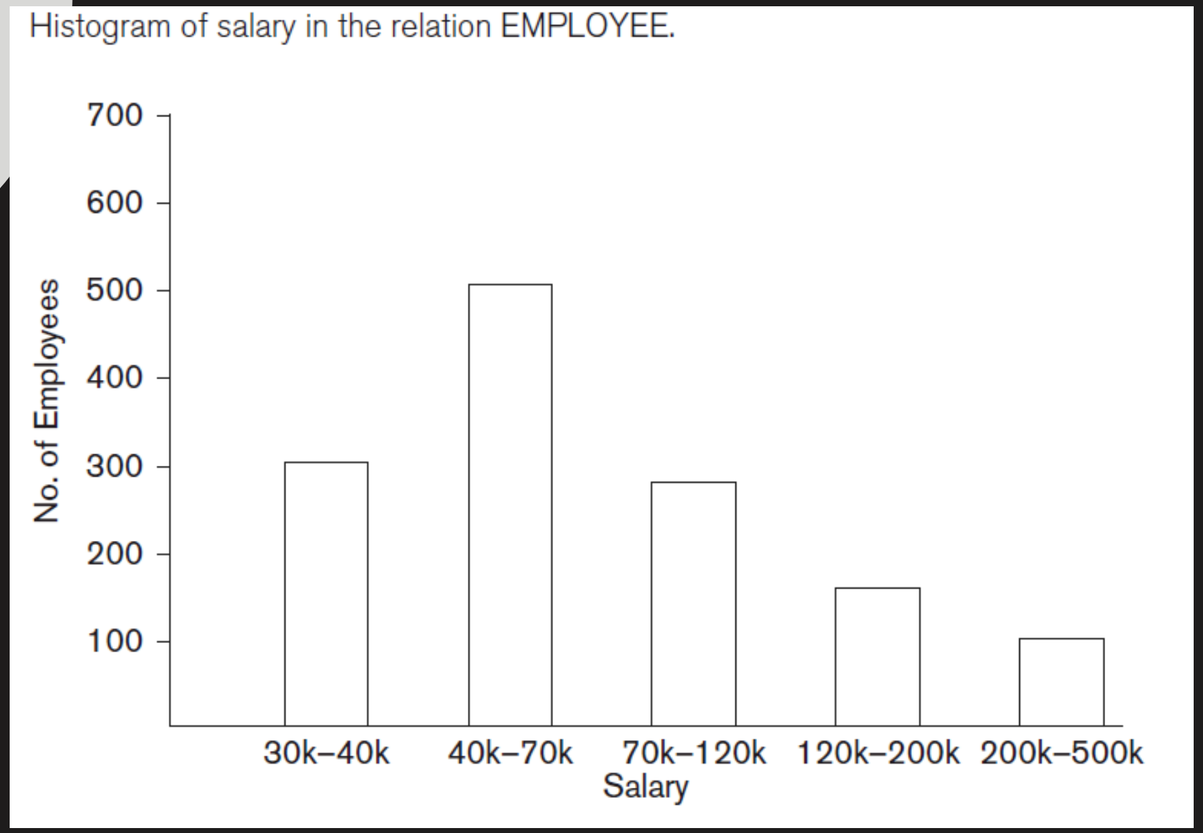
*Blevel is the number of levels without the leaf level.

Database Catalog

A database catalog is basically a metadata about data.
Typically there is one catalog per relation.
The catalog contains information, such as:

- Size of the file/files containing records/tuples for the relation.
- Number of records/tuples.
- Average record size.
- Number of file blocks.
- Blocking factor (number of records per file block)
- Index blocks and indexing structure.
- Number of distinct values for important attributes.
- Attribute Selectivity (fraction of records satisfying and equality condition on the attribute).
- Min and Max value for important attributes.
- etc.

Histogram



With database catalog, the system has to assume uniform distribution of records/tuples against interesting attributes used in the SQL queries. The uniform distribution assumption is too simplistic and often not so useful.

A frequency histogram charts the distribution of values of important attributes so that accurate estimate of selectivity of an attribute can be determined.

Single attribute histograms are common. However, sometimes histogram for attribute sets or functions over attributes are also retained if needed.

Technique of Cost-based Optimization

- The database management system establish a cost function that is a weighted sum of disk access, buffer consumption, computation complexity, storage overhead for intermediate results, and communication cost.

- Candidate alternative query trees from algebraic modifications are evaluated against this cost function using information from the database catalogs and histograms and the estimated best query is selected for execution.

- However the search space of alternative query tree can be exponential!!!

- So the cost-based optimizer applies dynamic programming and heuristic based pruning to explore a reasonable number of alternatives.

Database catalogs and histograms need to be kept up to date for high quality cost estimation. Which is done following some designated schedule.

Example

How many different ways join among four relations can be ordered?

- The two relations for the first join can be chosen in 4C2 = 6 ways.
- The two relations for the second join can be chosen in 3C2 = 3 ways.
- That fixes the relations for the last join.
- So overall ordering possibility is 6 * 3 = 18.

That is just the ordering of joins!!

For each join the implementation strategy can be:

1. Sort-merge join.
2. Regular nested-loop join.
3. Partitioned hash-join.
4. Index-based nested loop join.

So 18 * 4 = 72 alternative evaluation strategies.

Then the joins can be pipelined where an earlier join supplies tuples to a later join as a stream or the result of earlier joins can be stored (called materialization) in intermediate files. Giving 72 * 2 = 144 alternatives!!!

Combine joins with choices for select, project, and group by operations. Then you understand why the search space of query trees is so large.

Physical Optimizations

Physical optimization refers to the alternative ways the final query tree selected by the rule and cost-based optimizer can be processed. Options can be:

- bottom-up or top-down execution of parts from the query tree.
- Intermediate results sharing among inter/intra-query parts.
- Parallel executions using threads/processes.
- Query plan caching.
- Etc.

Query optimization is the most active field of research in the area of database systems due to its complexity and importance.

What Next?

LECTURE 13:

...



LOADING.....